



Assignment 1

Student Name: Kanishk Bhandari
Branch: BE CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS11100
Section/Group: 23BCS_KRG-2_B
Date: 04/02/2026
Subject Code: 23CSH-314

Q1. Explain SRP and OCP in detail with proper examples.

Ans. Single Responsibility Principle (SRP) states that a class should have only one responsibility or only one reason to change. This means that a class should focus on a single functionality and should not handle multiple unrelated tasks.

When a class performs more than one responsibility, any change in one responsibility may affect the others, leading to increased complexity and higher chances of errors. SRP promotes high cohesion and makes the system easier to maintain, test, and modify.

Example:

Consider a class that conducts the exam, evaluates answers, and generates results.

```
class OnlineExam {  
    void conductExam() { }  
    void evaluateAnswers() { }  
    void generateResult() { }  
}
```

This class violates SRP because it has multiple responsibilities. Any change in database logic or printing format requires modifying the same class.

To follow SRP, responsibilities are separated:

```
class ExamManager {  
    void conductExam() { }  
}  
  
class ExamEvaluator {  
    void evaluateAnswers() { }  
}  
  
class ResultGenerator {  
    void generateResult() { }  
}
```



Open/Closed Principle (OCP) states that software entities should be open for extension but closed for modification. This means new functionality can be added by extending existing code rather than changing it.

In an Online Examination System, multiple exam types such as MCQ exams, descriptive exams, or coding exams may be introduced. Modifying existing evaluation logic every time a new exam type is added violates OCP.

Example:

```
class ExamEvaluator {  
    void evaluate(String examType) {  
        if (examType.equals("MCQ")) { }  
        else if (examType.equals("Descriptive")) { }  
    }  
}
```

Adding a new exam type requires modifying the evaluator class, which violates OCP.

OCP-compliant Design:

```
interface ExamEvaluation {  
    void evaluate();  
}
```

```
class MCQExam implements ExamEvaluation {  
    public void evaluate() { }  
}
```

```
class DescriptiveExam implements ExamEvaluation {  
    public void evaluate() { }  
}
```

```
class ExamEvaluator {  
    void evaluateExam(ExamEvaluation exam) {  
        exam.evaluate();  
    }  
}
```

New exam types can be added by creating new classes without modifying existing code.

Q2. Discuss in detail about the violations in SRP and OCP along with their fixes.

Ans. Violations of Single Responsibility Principle (SRP)

The Single Responsibility Principle states that a class should have only one responsibility or one reason to change. A violation occurs when a single class performs multiple unrelated tasks.

Example of SRP Violation (E-Commerce Application)-

Consider an Order class that processes orders, calculates bills, and manages shipment details.

```
class Order {  
    void placeOrder() { }  
    void calculateBill() { }  
    void arrangeShipment() { }  
}
```

This class violates SRP because:

- Order processing rules may change
- Pricing or tax rules may change
- Shipping process may change independently

Thus, the class has multiple reasons to change.

Fix for SRP Violation

Responsibilities are separated into individual classes.

```
class OrderProcessor {  
    void placeOrder() { }  
}
```

```
class BillingService {  
    void calculateBill() { }  
}
```

```
class ShippingService {  
    void arrangeShipment() { }  
}
```

Each class now has a single responsibility, improving maintainability and clarity.

Violation of Open/Closed Principle (OCP)

OCP states that software entities should be open for extension but closed for modification. A violation occurs when existing code must be modified to support new features.

Example of OCP Violation (E-Commerce Application)

Consider a payment processing class that uses conditional logic for payment modes.

```
class PaymentProcessor {  
    void pay(String paymentType) {  
        if (paymentType.equals("Card")) { }  
        else if (paymentType.equals("UPI")) { }  
    }  
}
```

Adding a new payment mode like **Wallet** or **Net Banking** requires modifying the class, violating OCP.

Fix for OCP Violation

Using abstraction and polymorphism solves this issue.

```
interface PaymentMethod {  
    void pay();  
}  
  
class CardPayment implements PaymentMethod {  
    public void pay() { }  
}  
  
class UPIPayment implements PaymentMethod {  
    public void pay() { }  
}  
  
class PaymentProcessor {  
    void processPayment(PaymentMethod method) {  
        method.pay();  
    }  
}
```

New payment methods can now be added without modifying existing code.

Q3. Design an HLD for an Online Examination System applying these principles.

Ans. Now we'll be designing the High Level Design (HLD) of an Online Examination System using principles of SRP and OCP, which illustrates the major system modules and their interactions. The architecture follows a modular approach to support maintainability and future extensions.

1- Requirements: Functional & Non-Functional

A- Functional Requirement

- The system should allow users (students, admins, evaluators) to register and log in securely.

- Admin should be able to create exams, define duration, total marks, and schedule.
- Admin should be able to add different types of questions such as MCQ, descriptive, and coding questions.
- Students should be able to view available exams and start the exam within the allowed time window.
- The system should conduct exams in real time, manage timers, and collect responses.
- The system should evaluate answers using auto-evaluation, manual evaluation, or code evaluation.
- The system should generate and store results after evaluation.
- Students should be able to view their results after exam completion.

B- Non-Functional Requirement

- The system should support thousands of students simultaneously during peak exam times.
- Exam start and submission response time should be < 200 ms.
- Result fetching should be optimized.
- The system should be highly available during exams.
- Exam submissions and evaluations must be strongly consistent.

The system primarily favors Consistency over Availability for exam submission and evaluation, while result viewing favors Availability using caching and replicas.

2- Core-entities of System

- User
- Exam
- Question
- Answer/submission
- Evaluation
- Result

3- API endpoint creation

A. User API's

1. User Registration POST /api/users/register
2. User Login POST /api/users/login
3. User Profile Display GET /api/users/{user_id}/profile
User Profile Update PUT /api/users/{user_id}/profile

B. Exam API's

1. Create Exam- POST /api/exams
2. Update Exam Details - PUT /api/exams/{exam_id}
3. Delete Exam - DELETE /api/exams/{exam_id}
4. Get All Exams (Pagination) - GET /api/exams?limit={limit}&offset={offset}
5. Get Exam Details - GET /api/exams/{exam_id}

C. Question API's

1. Add Question to Exam- POST /api/exams/{exam_id}/questions
2. Update Question - PUT /api/questions/{question_id}
3. Delete Question - DELETE /api/questions/{question_id}
4. Get Questions for Exam - GET /api/exams/{exam_id}/questions

D. Exam Conduction API's

1. Start Exam - POST /api/exams/{exam_id}/start
2. Submit Exam - POST /api/exams/{exam_id}/submit
3. Auto Save Answers (Optional) - PUT /api/exams/{exam_id}/autosave

E. Evaluation API's

1. Update Evaluation Status - PUT /api/evaluations/{evaluation_id}/status

F. Result Management API's

1. Get Result for Exam - GET /api/results/{exam_id}
2. Get All Results for User - GET /api/users/{user_id}/results?limit={limit}&offset={offset}
3. Publish Result (Admin) - POST /api/results/{exam_id}/publish

4- High-Level Design

